

Dokumentacja Końcowa Projektu: Klasyfikacja Elementów Gitarowych przy użyciu Głębokiego Uczenia Maszynowego

Autor: kubuk224-cyber

9 czerwca 2026

Spis treści

1	Wstęp i Cel Projektu	3
2	Historia i Chronologia Rozwoju Projektu	3
3	Wykorzystane Technologie i Metodologia	4
3.1	Uczenie Transferowe (Transfer Learning)	4
3.2	Zaawansowana Augmentacja Danych	4
4	Ewolucja Kodu: Co Zmieniono i Usprawniono?	4
4.1	Błąd Nadpisywania Transformacji (Data Leakage)	4
4.2	Problem Wielkości Liter w Systemie Linux (Case-Sensitivity)	4
4.3	Zjawisko Overfittingu Mostków	5
4.4	Błąd Kadrowania Zdjęcia Testowego (Aspect Ratio Bug)	5
4.5	Kluczowy Przełom: Skalowanie Rozdzielczości do 600×600 px	5
5	Integracja i Rezultaty Końcowe	5
6	Podsumowanie i Wnioski	6

1 Wstęp i Cel Projektu

Celem projektu było stworzenie wielomodelowego systemu klasyfikacji (ang. *Pipeline Classification Model*), który na podstawie jednego zdjęcia wejściowego gitary elektrycznej potrafi poprawnie zidentyfikować:

1. **Kształt korpusu** (klasy: `single_cut`, `double_cut`),
2. **Rodzaj zastosowanego mostka** (klasy: `Floyd_style`, `Hardtail_style`, `Tremolo_style`, `Tune_o_matic_style`),

Projekt opiera się na bibliotece **PyTorch** i wykorzystuje technikę **Transfer Learningu** (Uczenia Transferowego) na bazie sieci ResNet pre-trenowanych na zbiorze ImageNet.

2 Historia i Chronologia Rozwoju Projektu

Na podstawie historii repozytorium GitHub oraz etapów prac, rozwój projektu przebiegał w sposób przyrostowy, ewoluując od prostego klasyfikatora binarnego do zaawansowanego systemu potokowego:

Data (2026)	Commit GitHub	Kamień Milowy / Wykorzystane Rozwiązanie
16 maja	<i>first work with guitar classifier</i>	Budowa podstawowej sieci binarnej (ResNet-18) do rozróżniania typów korpusu.
16 maja	<i>increased epochs...</i>	Zwiększenie liczby epok i zebranie większej bazy zdjęć w celu stabilizacji wag.
24 maja	<i>Changed into resnet50 model</i>	Migracja z ResNet-18 na ResNet-50 w celu zwiększenia pojemności sieci dla trudniejszych detali.
25 maja	<i>95% validation on 7 epochs</i>	Osiągnięcie wysokiej dokładności walidacyjnej dla korpusów we wczesnym etapie treningu.
1 czerwca	<i>more dataset</i>	Rozbudowa datasetu przy użyciu dedykowanego skryptu <code>scraper.py</code> (zbieranie zdjęć z Thomanna).
5 czerwca	<i>added guitar bridge ML</i>	Separacja zadań. Implementacja dedykowanego modelu dla mostków gitarowych.
5 czerwca	<i>added bridges</i>	Przejsięcie na architekturę Custom Dataset z pominięciem folderów korpusów.
Finał	<i>Resolution Scale & Fine-Tuning</i>	Wzrost do 95% (korpusy) i 97% (mostki) dzięki zmianie rozdzielczości na 600×600 px i podwójnemu dropoutowi.

Tabela 1: Chronologia rozwoju projektu na podstawie wpisów z repozytorium i etapów optymalizacji.

3 Wykorzystane Technologie i Metodologia

3.1 Uczenie Transferowe (Transfer Learning)

W projekcie zrezygnowano z trenowania sieci od zera z powodu ryzyka przeuczenia przy małej liczbie próbek. Wykorzystano modele:

- **ResNet-18:** Wykorzystany pierwotnie do rozpoznawania cech makroskopowych (ogólny kontur korpusu).
- **ResNet-50:** Zastosowany do ostatecznej klasyfikacji mikro-detali (mostki gitarowe), oferujący bogatszą reprezentację wektorową (2048 cech wejściowych klasyfikatora zamiast 512).

3.2 Zaawansowana Augmentacja Danych

W celu sztucznego zwiększenia datasetu i uodpornienia sieci na zaszumione zdjęcia użytkowników, w skryptach zaimplementowano transformacje `torchvision.transforms`:

- Losowe wycinanie i przeskalowywanie (`RandomResizedCrop`),
- Odbicia lustrzane (`RandomHorizontalFlip`),
- Losowa rotacja (`RandomRotation`) ograniczona do 15–25 stopni,
- Manipulacja kolorami, jasnością i kontrastem (`ColorJitter`),
- **Autorski Szum Gaussa (`AddGaussianNoise`):** Klasa dodająca losowe zakłócenia bezpośrednio na tensory przed normalizacją.

Zbiór walidacyjny został pozbawiony powyższych transformacji.

4 Ewolucja Kodu: Co Zmieniono i Usprawniono?

W trakcie prac nad kodem zidentyfikowano i wyeliminowano szereg krytycznych błędów architektonicznych:

4.1 Błąd Nadpisywania Transformacji (Data Leakage)

Co było nie tak: W pierwotnej implementacji `random_split` dokonano podziału na podzbiory, a następnie wywołano instrukcję `val_dataset.dataset.transform = val_transforms`. Ponieważ w PyTorch podzbiory współdzielały referencję do tego samego bazowego datasetu, instrukcja ta cicho nadpisywała transformacje treningowe.

Jak to zmieniono: Zaimplementowano podwójne niezależne instancje klasy datasetu spięte generatorem z identycznym ziarnem losowości (`manual_seed(42)`).

4.2 Problem Wielkości Liter w Systemie Linux (Case-Sensitivity)

Co było nie tak: Program zgłaszał błąd `ValueError: num_samples should be a positive integer`.

Jak to zmieniono: Zdiagnozowano, że system Fedora rygorystycznie podchodzi do wielkości znaków. Zaktualizowano listę `ALLOWED_CLASSES`, dopasowując ją do rzeczywistych nazw folderów na dysku (np. `Single_cut`).

4.3 Zjawisko Overfittingu Mostków

Co było nie tak: Model początkowo uzyskiwał wysoką dokładność na danych treningowych (75%), lecz na walidacji osiągał jedynie 55%.

Jak to zmieniono: Wprowadzono głęboki *Fine-Tuning* (odmrożenie bloku `layer4` w ResNet-50) oraz zróżnicowane współczynniki uczenia (*Discriminative Learning Rates*). Zastosowano również agresywny, dwustopniowy `nn.Dropout(p=0.5)` oraz mechanizm równoważenia klas (`Class Weights`).

4.4 Błąd Kadrowania Zdjęcia Testowego (Aspect Ratio Bug)

Co było nie tak: W ostatecznym skrypcie integrującym (`finale.py`) model błędnie klasyfikował podłużne zdjęcia gitar (często wyrzucając fałszywe wyniki np. *double_cut* dla Les Paula z powodu `CenterCrop(224)`, który ucinął mostek i rogi gitary).

Jak to zmieniono: Wymuszono sztywne przeskalowanie geometryczne bez ucinania krawędzi (`Resize((600, 600))`).

4.5 Kluczowy Przełom: Skalowanie Rozdzielczości do 600×600 px

Usprawnienie: Aby dostrzec precyzyjne elementy mechaniczne (np. śrubki we wózkach mostków Floyd Rose vs Tune-o-matic), zwiększono natywny rozmiar wejściowy sieci ze standardowych 224 do 600 pikseli. Bezpiecznie zmniejszono rozmiar paczki (`BATCH_SIZE`), chroniąc kartę graficzną przed błędem `CUDA Out of Memory`.

Rezultat: Dostarczenie sieci znacznie większej ilości pikseli pozwoliło na błyskawiczny i spektakularny wzrost skuteczności, eliminując wcześniejsze problemy z brakiem widoczności detali. **To ulepszenie zaowocowało ostatecznymi wynikami na poziomie 95% dla korpusów i aż 97% dla mostków.**

5 Integracja i Rezultaty Końcowe

Ostateczny skrypt `finale.py` wczytuje niezależnie oba wyspecjalizowane modele. Dzięki ujednoliceniu rozdzielczości do 600×600 px, potok przetwarzania generuje prawidłowe, sprzężone odpowiedzi, które nanoszone są na analizowane zdjęcie. Natomiast z powodu ograniczonego datasetu i niewielkiej ilości klas, nie wszystkie modele gitar zostają rozpoznawane prawidłowo.

Analiza Gitary AI



Kształt: single_cut (93.5%)

Mostek: Hardtail_style (99.8%)

Rysunek 1: Wizualizacja graficzna końcowej predykcji połączonych modeli po wprowadzeniu poprawek w transformacjach i zwiększeniu rozdzielczości do 600x600.

Analiza Gitary AI



Kształt: double_cut (51.7%)

Mostek: Floyd_style (66.6%)

Rysunek 2: Wizualizacja graficzna końcowej predykcji połączonych modeli dla gitary nie będącej zdefiniowana w klasach. Wynik dla modelu jest niepoprawny.

6 Podsumowanie i Wnioski

Projekt udowodnił, że podział skomplikowanego zagadnienia wizyjnego na wyspecjalizowane pod-modele (modułowa architektura *Pipeline*) pozwala na optymalne dopasowanie

parametrów do konkretnego zadania.

Kluczem do ostatecznego sukcesu okazało się odejście od domyślnej rozdzielczości standardu ImageNet (224×224) na rzecz obrazów o wysokiej detaliczności (600×600). Zastosowanie technik takich jak *AdamW*, *Cosine Annealing*, asymetryczny fine-tuning starych warstw oraz silny Dropout pozwoliło wyeliminować przeuczenie. **W finalnej wersji system uzyskał fenomenalne 95% dokładności na zbiorze walidacyjnym w rozpoznawaniu kształtów gitar (Single-cut vs Double-cut) oraz aż 97% skuteczności w trudniejszym zadaniu identyfikacji skomplikowanych układów mostków.**